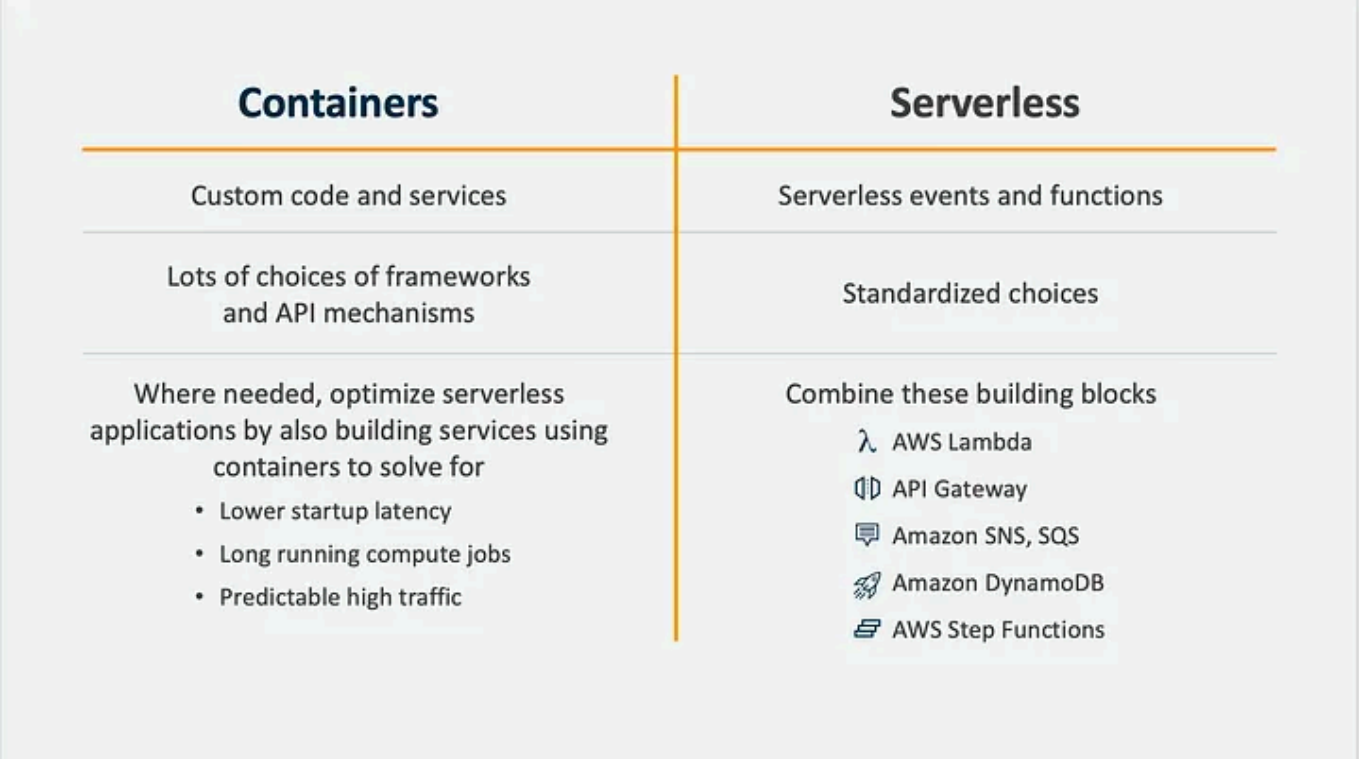
Get unlimited access to the best of Medium for less than $1/week. Become a member

So many bad takes — What is there to learn from the Prime Video microservices to monolith story

adrian cockcroft Follow 4 min read · May 6, 2023

859 9



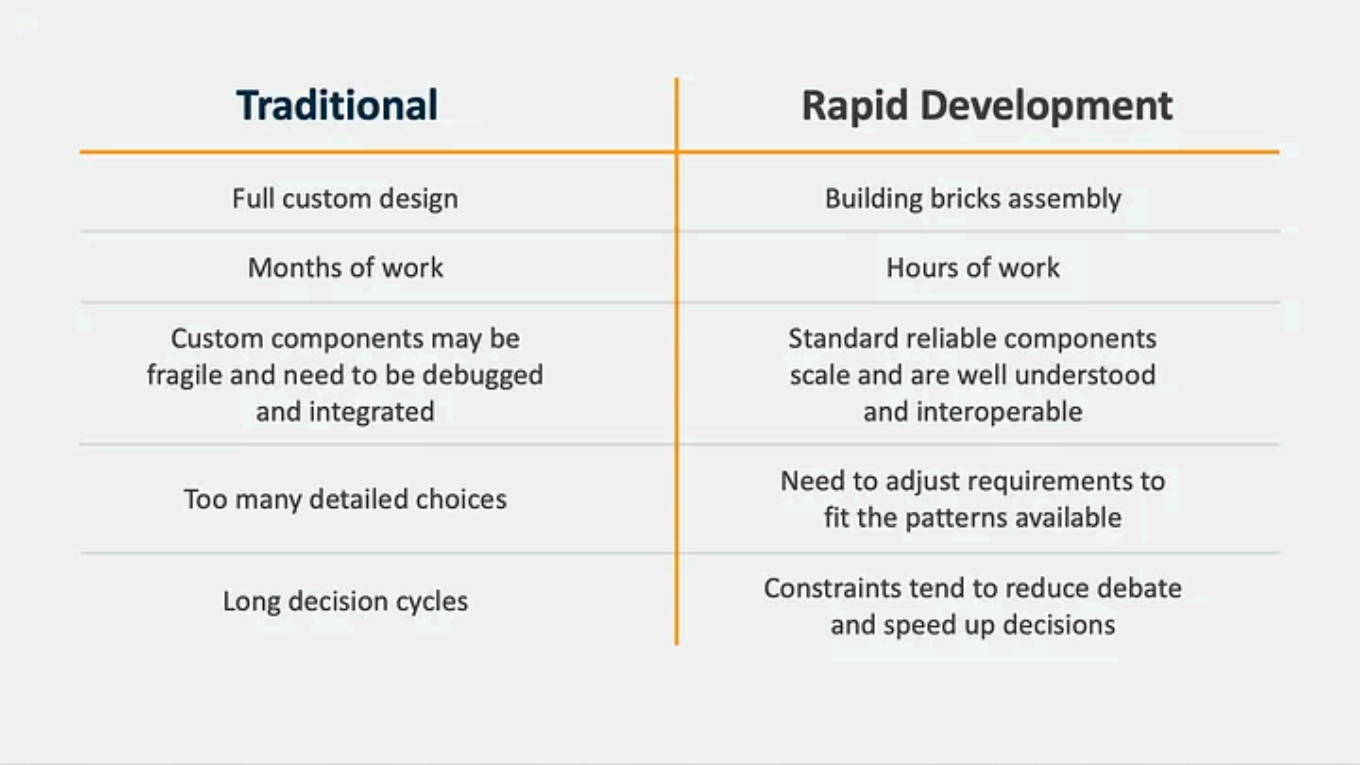
Excerpt from Serverless First deck first published in 2019

The Prime Video team published this story: Scaling up the audio/video monitoring service and reducing costs by 90%, and the internet piled in with opinions and bad takes, mostly missing the point. What the team did follows the advice I've been giving for years (here's a video from 2019):

> *"Where needed, optimize serverless applications by also building services using containers to solve for lower startup latency, long running compute jobs, and predictable high traffic"*

The Prime Video team had followed a path I call Serverless First, where the first try at building something is put together with Step Functions and Lambda calls. They state in the blog that this was quick to build, which is the point. When you are exploring how to construct something, building a prototype in a few days or weeks is a good approach. Then they tried to scale it to cope with high traffic and discovered that some of the state transitions in their step functions were too frequent, and they had some overly chatty calls between AWS lambda functions and S3. They were able to re-use most of their working code by combining it into a single long running microservice that is horizontally scaled using ECS, and which is invoked via a lambda function. This is only one of many microservices that make up the Prime Video application. The problem is that they called this refactoring a microservice to monolith transition, when it's clearly a microservice refactoring step, and is exactly what I recommend people do in my talks about Serverless First. I don't advocate "Serverless Only", and I recommended that if you need sustained high traffic, low latency and higher efficiency, then you should re-implement your rapid prototype as a continuously running autoscaled container, as part of a larger serverless event driven architecture, which is what they did. If you built it as a microservice to start with, it would probably take longer (especially as you

have to make lots of decisions about how to build and run it), and be less able to iterate as you figure out exactly what you are trying to build.



Traditional	Rapid Development
Full custom design	Building bricks assembly
Months of work	Hours of work
Custom components may be fragile and need to be debugged and integrated	Standard reliable components scale and are well understood and interoperable
Too many detailed choices	Need to adjust requirements to fit the patterns available
Long decision cycles	Constraints tend to reduce debate and speed up decisions

Excerpt from Serverless First deck published in 2019

In contrast to commentary along the lines that Amazon got it wrong, the team followed what I consider to be the best practice. The result isn't a monolith, but there seems to be a popular trigger meme nowadays about microservices being over-sold, and a return to monoliths. There is some truth to that, as I do think microservices were over sold as the answer to everything, and I think this may have arisen from vendors who wanted to sell Kubernetes with a simple marketing message that enterprises needed to modernize by using Kubernetes to do cloud native microservices for everything. What we are seeing is a backlash to that messaging, and a realization that the complexity of Kubernetes has a cost, which you don't need unless you are running at scale with a large team. Ironically, many enterprise workloads are intermittent and small scale and very good

candidates for a serverless first approach using Step Functions and Lambda. See The Value Flywheel Effect book for more on serverless first, and read Sam Newman's Building Microservices: Desiging Fine-Grained Systems book to get the best practices on when and how to use the techniques to effectively build, manage and operate this way. His first edition in 2015 was foundational, and he updated it in 2021 with a second edition. He is also clear about when microservices aren't useful.



Finally, what were they building? A real-time user experience analytics engine for live video, that looked at all users rather than a subsample. This is a very good thing to have, in fact Netflix built in monitoring for all users at the start of it's streaming launch in 2007, and it was the very first workload that moved to AWS in 2009. Now that Netflix has also added live broadcasts, I assume they've extended their own capabilities to do something similar to what Prime Video describes. If you happen to be running a video streaming service and don't have real time user experience monitoring built in to your architecture, I suggest you take a look at Datazoom.io which provides this as a service and where the chief architect and CTO are both ex-Netflix

Open in app ↗

Medium Search Write

AWS + Prime Video + Monolithic Architecture + Microservices + Serverless +



Written by adrian cockcroft

13.2K followers · 465 following

Follow

Work: Technology strategy advisor, Partner at OrionX.net (ex Amazon Sustainability, AWS, Battery Ventures, Netflix, eBay, Sun Microsystems, CCL)

Responses (9)



Francisco Moreno

What are your thoughts?



Aviran Mordo
May 7, 2023

I tend to agree with you. At Wix we have taken Microservices and Serverless a step further, combining the two and creater Platform as a Runtime (Paar), which allows developers to develop as functions or microservices, but run an in optimized way as... more

11 Reply



Hadosson XLIX
May 8, 2023

I thought people have to understand the business application they are working with. People take Netflix or Dropbox approach to solve an Insurance Claims system. The database design and type is important. If you care about transactions managed by the... more

6 Reply



Joshua Szepietowski
May 7, 2023

Fantastic article. It's good to see a balanced response to the knee jerk reactions I've been seeing across the interwebs.

5 Reply

See all responses

More from adrian cockcroft



adrian cockcroft · 3d ago

You all needed to read Output to Outcome eight months ago...

I've known Mik Kersten for a while, his 2018 book Project to Product and the whole...

5 1



adrian cockcroft · Jun 4

How reliable, fast and expensive is each version of Claude Code...

[Updated for Fable-5] Every few weeks a new frontier model tops the leaderboards, and th...

16 2



adrian cockcroft · Apr 9

Entering NoMan's Land

Photo of kangaroo paw flower by Adrian

56 3



adrian cockcroft · Jun 26, 2025

Vibe Coding is so "Last Month..." — My First Agent Swarm Experience...

I've been reading stories posted on LinkedIn by Reuven Cohen, and he's been using agent...

153 10

See all from adrian cockcroft